Day 2 - Strings Palindrome & Anagram

ITSRUNTYM

Topics: Palindromes and Anagrams (with LeetCode examples)

I. PALINDROME

1. Definition

A string is a **palindrome** if it reads the same forward and backward.

- Examples:
  - "madam" → palindrome
  - "racecar" → palindrome
  - "hello" → not a palindrome

2. Types of Palindromes

Type	Description	Example
Normal Palindrome	Exact match	"abba"
Case-insensitive	Ignores case	"AbBa"
Alphanumeric only	Ignores non-alphanumeric characters	"A man, a plan, a canal: Panama"

3. Approaches

Two Pointers Approach

- Start one pointer at the beginning, and one at the end.
- Compare characters; move pointers inward if match.

Reverse and Compare

- Reverse the string and compare with original.

4. LeetCode Questions

LC 125. Valid Palindrome

Easy

Input: "A man, a plan, a canal: Panama"
Output: true

Approach:

```
class Solution {
    public boolean isPalindrome(String s) {
        s = s×toLowerCase();
        s= s×replaceAll("[^a-z0-9]","");
        int n = s.length();
        for(int i=0;i<n/2;i++){
            if(s×charAt(i) != s.charAt(n-1-i))
            return false;
        }
        return true;
    }
}
```

Time: O(n), Space: O(1)

C++ Version

```
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>
#include <regex>

class Solution {
public:
    bool isPalindrome(std::string s) {
        // Convert to lowercase
        std::transform(s.begin(), s.end(), s.begin(), ::tolower);

        // Remove non-alphanumeric characters using regex
        s = std::regex_replace(s, std::regex("[^a-z0-9]"), "");

        int n = s.length();
        for (int i = 0; i < n / 2; i++) {
            if (s[i] != s[n - 1 - i])
```

```
            return false;
        }
        return true;
    }
};
```

LC 680. Valid Palindrome II

Easy

Input: "abca" → Can delete one character to make it palindrome
Approach: Use two pointers. On mismatch, try skipping either character.

Optimized Iterative Approach (No Recursion)

Idea

- Use **two pointers** (left, right).
- When you find a mismatch, try:
  - skipping the left character → isPalindromeRange(s, left+1, right)
  - skipping the right character → isPalindromeRange(s, left, right-1)
- If either is a palindrome, return true.

Code

```
public boolean validPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        if (s×charAt(left) != s.charAt(right)) {
            // Try skipping either left or right character
            return isPalindromeRange(s, left + 1, right) || isPalindromeRange(s, left,
right - 1);
        }
        left++;
        right--;
    }
    return true; // It's already a palindrome
}
// Helper to check if s[left..right] is a palindrome
private boolean isPalindromeRange(String s, int left, int right) {
    while (left < right) {
        if (s×charAt(left) != s.charAt(right)) return false;
        left++;
        right--;
```

```
    }
    return true;
}
```

Recursion

```
public boolean validPalindrome(String s) {
    return isPalin(s, 0, s.length() - 1, false);
}
private boolean isPalin(String s, int left, int right, boolean deleted) {
    while (left < right) {
        if (s×charAt(left) != s.charAt(right)) {
            if (deleted) return false;
            return isPalin(s, left + 1, right, true) || isPalin(s, left, right - 1, true);
        }
        left++; right--;
    }
    return true;
}
```

C++ Code

```
#include <iostream>
#include <string>
class Solution {
public:
    bool validPalindrome(std::string s) {
        int left = 0, right = s.length() - 1;
while (left < right) {
            if (s[left] != s[right]) {
                // Try skipping either the left or the right character
                return isPalindromeRange(s, left + 1, right) || isPalindromeRange(s,
left, right - 1);
            }
            left++;
            right--;
        }
        return true; // It's already a palindrome
    }
private:
    bool isPalindromeRange(const std::string& s, int left, int right) {
        while (left < right) {
            if (s[left] != s[right]) return false;
            left++;
```

```
            right--;
        }
        return true;
    }
};
```

C++ Code (Recursive Version)

```
#include <iostream>
#include <string>
class Solution {
public:
    bool validPalindrome(std::string s) {
        return isPalin(s, 0, s.length() - 1, false);
    }
private:
    bool isPalin(const std::string& s, int left, int right, bool deleted) {
        while (left < right) {
            if (s[left] != s[right]) {
                if (deleted) return false;
                return isPalin(s, left + 1, right, true) || isPalin(s, left, right - 1, true);
            }
            left++;
            right--;
        }
        return true;
    }
};
```

II. ANAGRAM

1. Definition

Two strings are **anagrams** if they contain the same characters with the same frequency, **order doesn't matter**.

- Example:
  "listen" and "silent" →
  "aabb" and "abab" →
  "rat" and "car" →

2. Approaches

Sorting
Sort both strings and compare.

HashMap / Frequency Array
Count frequencies and compare.

3. LeetCode Questions

LC 242. Valid Anagram

Easy

Check if two strings are anagrams.

Approach: Frequency array

```
public boolean isAnagram(String s, String t) {
    if (s×length() != t.length())
            return false;
    int[] count = new int[26];
    for (char c : s.toCharArray())
          count[c - 'a']++;
    for (char c : t.toCharArray()) {
        if (--count[c - 'a'] < 0) return false;
    }
    return true;
}
```

Java Code Using HashMap

```
import java.util.*;
public class Solution {
    public boolean isAnagram(String s, String t) {
        if (s×length() != t.length()) return false;
Map<Character, Integer> freqMap = new HashMap<>();
// Count frequency of characters in s
        for (char c : s.toCharArray()) {
            freqMap.put(c, freqMap.getOrDefault(c, 0) + 1);
        }
// Subtract frequency based on characters in t
        for (char c : t.toCharArray()) {
            if (!freqMap.containsKey(c))
                    return false;
            freqMap.put(c, freqMap.get(c) - 1);
            if (freqMap.get(c) < 0)
                    return false;
        }
// Optional: All values should be 0 now
        // (This step is redundant if the lengths match and above passed)
        return true;
    }
}
```

C++ Version

```
#include <string>
using namespace std;

class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s×length() != t.length())
            return false;
```

```
        int count[26] = {0};

        for (char c : s)
            count[c - 'a']++;

        for (char c : t) {
            if (--count[c - 'a'] < 0)
                return false;
        }

        return true;
    }
};
```

C++ Code (Using unordered_map):

```
#include <string>
#include <unordered_map>
using namespace std;
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s×length() != t.length())
            return false;
unordered_map<char, int> freqMap;
// Count frequency of characters in s
        for (char c : s) {
            freqMap[c]++;
        }
// Subtract frequency based on characters in t
        for (char c : t) {
            if (freqMap×find(c) == freqMap.end())
                return false;
freqMap[c]--;
if (freqMap[c] < 0)
                return false;
        }
// Optional: All values should be 0 now
        // (This check is redundant because of length check and logic above)
return true;
    }
};
```

LC 49. Group Anagrams

Medium

Group strings that are anagrams of each other.

Approach: Use a HashMap with sorted string as key.

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        //sorted string as key , and other anagrams in the arraylist
        HashMap<String, List<String>> map = new HashMap<>();
        for(String s : strs){
            char[] arr = s×toCharArray();
            Arrays.sort(arr);
            String str1 = new String(arr);
            if(!map.containsKey(str1)){
                map.put(str1, new ArrayList<String>());
            }
            map.get(str1).add(s); //anagrams
        }
        return new ArrayList<>(map.values());
    }
}
```

C++ Version

```
#include <vector>
```

```
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        // Map sorted string to list of anagrams
        unordered_map<string, vector<string>> map;

        for (const string& s : strs) {
            string sortedStr = s;
            sort(sortedStr.begin(), sortedStr.end());  // Sort the characters

            map[sortedStr].push_back(s);  // Add original string to its group
        }

        // Collect grouped anagrams from the map
        vector<vector<string>> result;
        for (auto& pair : map) {
            result.push_back(pair.second);
        }

        return result;
    }
};
```

Palindrome Based problems

No.	**Problem Name**	**Difficulty**	**Link**
1	Valid Palindrome	Easy	LC 125
2	Valid Palindrome II	Easy	LC 680
3	Longest Palindromic Substring	Medium	LC 5
4	Palindromic Substrings	Medium	LC 647
5	Palindrome Partitioning	Medium	LC 131

Anagram Based Problems

No.	Problem Name	Difficulty	Link
6	Valid Anagram	Easy	LC 242
7	Group Anagrams	Medium	LC 49
8	Find All Anagrams in a String	Medium	LC 438
9	Permutation in String	Medium	LC 567
10	Minimum Number of Steps to Make Two Strings Anagram	Easy	LC 1347